PROGRAMACIÓN IV · UTN RECONQUISTA

Módulo *Materias*

Incorporación de un nuevo módulo al Sistema de Gestión Académica: modelo, API REST y pantalla en React.

PRG1 PRG4 BD1 ¿?

Alumno	Valentino Serafini
Carrera	Tecnicatura Universitaria en Programación
Docente	Prof. Irina Agretti
Modalidad	Trabajo práctico individual
Fecha	Octubre de 2026

00 Antes de empezar

El módulo Materias lo armé tomando como base lo que ya teníamos hecho en Alumnos y Docentes. La estructura del backend (modelo, controlador, rutas) y la del frontend (página, formulario y lista comunicados por props) son las mismas, así que el que conozca esos módulos va a reconocer el recorrido.

La diferencia más grande es el identificador. En Alumnos usamos el legajo y en Docentes el DNI, los dos numéricos. Acá la clave es `codigo`, un texto como `PRG4` o `BD1`, y todas las operaciones la usan en lugar del `_id` de MongoDB. Además le sumé más validaciones de las que tenían los módulos anteriores, porque la consigna las pedía en detalle.

Diseño de la entidad

CAMPO	TIPO	OBLIG.	REGLAS QUE APLIQUÉ
`codigo`	String	Sí	Único. Se recortan los espacios de los extremos y se guarda en mayúsculas. Máximo 15 caracteres.
`nombre`	String	Sí	Sin espacios en los extremos, entre 3 y 100 caracteres.
`descripcion`	String	No	Si se informa, no puede ser solo espacios. Máximo 300 caracteres.
`cargaHoraria`	Number	Sí	Entero mayor que cero.
`estado`	String	Sí	Solo `activa` o `inactiva`.

Archivos del trabajo

01 Backend: `models/Materia.js`, `controllers/materias.controller.js`, `routes/materias.routes.js` y lo agregado en `server.js`.

02 Pruebas de la API con Rest Client (`pruebas.http`).

03 Frontend: `pages/Materias.jsx`, `components/FormularioMateria.jsx`, `components/ListaMaterias.jsx`, y los cambios en `App.jsx` y `Header.jsx`.

04 Decisiones y problemas que encontré en el camino.

Mantuve los nombres de archivo que ya usa el proyecto (`alumnos.controller.js`, `docentes.routes.js`), por eso el controlador y las rutas se llaman `materias.controller.js` y `materias.routes.js` en vez de los nombres orientativos de la consigna.

01 Backend

models/Materia.js NUEVO

Define el esquema de Mongoose. Cada regla lleva su propio mensaje en español, así si alguna validación llega hasta la base el usuario recibe un texto entendible y no el error crudo de Mongoose. `unique: true` crea el índice único sobre `codigo`, que es la protección definitiva contra duplicados. Con `trim` y `uppercase`, `" prg4 "` y `PRG4` terminan guardándose igual y no pueden convivir como dos materias distintas. Para la carga horaria uso `min: 1` junto con un validador `Number.isInteger`, porque `min` solo no frena valores como 12.5.

backend/models/Materia.js

```
const mongoose = require("mongoose")

const materiaSchema = new mongoose.Schema({
    codigo: {
        type: String,
        required: [true, "El código es obligatorio"],
        unique: true,
        trim: true,
        uppercase: true,
        maxlength: [15, "El código no puede superar los 15 caracteres"]
    },
    nombre: {
        type: String,
        required: [true, "El nombre es obligatorio"],
        trim: true,
        minlength: [3, "El nombre debe tener al menos 3 caracteres"],
        maxlength: [100, "El nombre no puede superar los 100 caracteres"]
    },
    descripcion: {
        type: String,
        trim: true,
        maxlength: [300, "La descripción no puede superar los 300 caracteres"],
        default: ""
    },
    cargaHoraria: {
        type: Number,
        required: [true, "La carga horaria es obligatoria"],
        min: [1, "La carga horaria debe ser mayor que cero"],
        validate: {
            validator: Number.isInteger,
            message: "La carga horaria debe ser un número entero"
        }
    },
    estado: {
        type: String,
        required: [true, "El estado es obligatorio"],
        enum: {
            values: ["activa", "inactiva"],
            message: "El estado solo puede ser activa o inactiva"
        }
    }
},
{
    versionKey: false
```

```
}
)

const Materia = mongoose.model("Materia", materiaSchema)

module.exports = Materia
```

controllers/materias.controller.js NUEVO

Acá está la lógica de las cinco operaciones. Respeté la forma de los otros controladores (funciones `async` que responden `{ mensaje }` cuando algo sale mal) y le agregué tres funciones de apoyo:

- `normalizarCodigo` recorta y pasa a mayúsculas el código que llega por URL o en el cuerpo, para buscarlo igual que como se guardó.
- `validarMateria` revisa los datos antes de tocar la base: campos obligatorios, tipos (el código tiene que ser texto y la carga horaria un número, no `"64"`), cadenas de solo espacios, rango de la carga horaria y estado permitido. Devuelve el primer error que encuentra, o `null`.
- `manejarError` centraliza las respuestas de error: el código `11000` de MongoDB (clave duplicada) se transforma en un 409, un `ValidationError` en un 400 con el mensaje del esquema, y cualquier otra cosa en un 500 genérico. El detalle técnico solo se muestra en la consola del servidor.

Al crear, primero busco si el código ya existe para dar un mensaje claro. Igual, si dos pedidos llegan al mismo tiempo los dos podrían pasar esa búsqueda, y en ese caso el índice único rechaza al segundo y `manejarError` lo convierte en el mismo 409. Lo probé mandando cinco altas simultáneas con el mismo código: entró una y las otras cuatro respondieron 409.

Para modificar no usé `findOneAndUpdate` como en Alumnos. Busco la materia por el código de la URL, si el código nuevo es distinto verifico que no lo tenga otra materia, y después aplico los cambios con `set` y `save`. Así se ejecutan todas las validaciones del esquema y una materia que conserva su propio código no se toma como duplicada.

backend/controllers/materias.controller.js

```
const Materia = require("../models/Materia")

const ESTADOS = ["activa", "inactiva"]

function normalizarCodigo(codigo) {
    return String(codigo).trim().toUpperCase()
}

function validarMateria({ codigo, nombre, descripcion, cargaHoraria, estado }) {
    if (codigo === undefined || nombre === undefined || cargaHoraria === undefined || estado === undefined) {
        return "Código, nombre, carga horaria y estado son obligatorios"
    }
    if (typeof codigo !== "string") {
        return "El código debe ser un texto"
    }
    if (codigo.trim() === "") {
        return "El código no puede estar vacío"
    }
    if (typeof nombre !== "string" || nombre.trim() === "") {
        return "El nombre no puede estar vacío"
    }
```

```
    if (nombre.trim().length < 3 || nombre.trim().length > 100) {
        return "El nombre debe tener entre 3 y 100 caracteres"
    }
    if (descripcion !== undefined) {
        if (typeof descripcion !== "string") {
            return "La descripción debe ser un texto"
        }
        if (descripcion.length > 0 && descripcion.trim() === "") {
            return "La descripción no puede contener únicamente espacios"
        }
    }
    if (typeof cargaHoraria !== "number" || !Number.isInteger(cargaHoraria) || cargaHoraria <= 0) {
        return "La carga horaria debe ser un número entero mayor que cero"
    }
    if (!ESTADOS.includes(estado)) {
        return "El estado solo puede ser activa o inactiva"
    }
    return null
}

function manejarError(res, error) {
    if (error.code === 11000) {
        return res.status(409).json({
            mensaje: "Ya existe una materia con ese código"
        })
    }
    if (error.name === "ValidationError") {
        const primerError = Object.values(error.errors)[0]
        return res.status(400).json({
            mensaje: primerError.message
        })
    }
    console.error(error)
    res.status(500).json({
        mensaje: "No se pudo completar la operación en la base de datos"
    })
}

async function obtenerMaterias(req, res) {
    try {
        const materias = await Materia.find().sort({ codigo: 1 })
        res.json(materias)
    } catch (error) {
        manejarError(res, error)
    }
}

async function obtenerMateria(req, res) {
    try {
        const materia = await Materia.findOne({
            codigo: normalizarCodigo(req.params.codigo)
        })
        if (!materia) {
            return res.status(404).json({
                mensaje: "Materia no encontrada"
            })
        }
        res.json(materia)
    } catch (error) {
        manejarError(res, error)
    }
```

```
}

async function crearMateria(req, res) {
    const mensajeError = validarMateria(req.body)
    if (mensajeError) {
        return res.status(400).json({
            mensaje: mensajeError
        })
    }
    const { codigo, nombre, descripcion, cargaHoraria, estado } = req.body
    try {
        const existe = await Materia.findOne({
            codigo: normalizarCodigo(codigo)
        })
        if (existe) {
            return res.status(409).json({
                mensaje: "Ya existe una materia con ese código"
            })
        }
        const nuevaMateria = await Materia.create({
            codigo,
            nombre,
            descripcion,
            cargaHoraria,
            estado
        })
        res.status(201).json(nuevaMateria)
    } catch (error) {
        manejarError(res, error)
    }
}

async function actualizarMateria(req, res) {
    const mensajeError = validarMateria(req.body)
    if (mensajeError) {
        return res.status(400).json({
            mensaje: mensajeError
        })
    }
    const { codigo, nombre, descripcion, cargaHoraria, estado } = req.body
    try {
        const materia = await Materia.findOne({
            codigo: normalizarCodigo(req.params.codigo)
        })
        if (!materia) {
            return res.status(404).json({
                mensaje: "Materia no encontrada"
            })
        }
        const nuevoCodigo = normalizarCodigo(codigo)
        if (nuevoCodigo !== materia.codigo) {
            const existe = await Materia.findOne({ codigo: nuevoCodigo })
            if (existe) {
                return res.status(409).json({
                    mensaje: "Ya existe otra materia con ese código"
                })
            }
        }
        materia.set({
            codigo,
            nombre,
            descripcion: descripcion ?? "",
```

```
            cargaHoraria,
            estado
        })
        await materia.save()
        res.json(materia)
    } catch (error) {
        manejarError(res, error)
    }
}

async function eliminarMateria(req, res) {
    try {
        const materia = await Materia.findOneAndDelete(
            { codigo: normalizarCodigo(req.params.codigo) }
        )
        if (!materia) {
            return res.status(404).json({
                mensaje: "Materia no encontrada"
            })
        }
        res.json({ mensaje: "Materia eliminada correctamente" })
    } catch (error) {
        manejarError(res, error)
    }
}

module.exports = {
    obtenerMaterias,
    obtenerMateria,
    crearMateria,
    actualizarMateria,
    eliminarMateria
}
```

routes/materias.routes.js NUEVO

Asocia cada método HTTP con su función del controlador. A diferencia de Alumnos y Docentes, el parámetro se llama `:codigo` en vez de `:id`, para que quede claro que no se trata del `_id`.

backend/routes/materias.routes.js

```
const express = require("express")
const {
    obtenerMaterias,
    obtenerMateria,
    crearMateria,
    actualizarMateria,
    eliminarMateria } = require("../controllers/materias.controller")
const router = express.Router()

router.get("/", obtenerMaterias)

router.get("/:codigo", obtenerMateria)

router.post("/", crearMateria)

router.put("/:codigo", actualizarMateria)

router.delete("/:codigo", eliminarMateria)
```

```
19
20 module.exports = router
```

server.js MODIFICADO

Solo agregué dos líneas: el `require` de las rutas, junto a los de Alumnos y Docentes, y el registro bajo el prefijo `/materias`.

backend/server.js *línea agregada*

```
9 const materiasRoutes = require("./routes/materias.routes")
```

backend/server.js *línea agregada*

```
17 app.use("/materias", materiasRoutes)
```

02 Pruebas de la API

Antes de pasar a React probé todo con Rest Client. Las peticiones están en `backend/pruebas.http`, numeradas igual que en la consigna. En cada caso miré el estado HTTP y la respuesta, y después volví a listar para confirmar que el cambio hubiera quedado guardado en MongoDB.

#	PRUEBA	PETICIÓN	ESTADO	RESULTADO
1	Listar con la colección vacía	`GET /materias`	200	Devuelve []
2	Registrar varias y listarlas	`POST + GET /materias`	201 / 200	Aparecen ordenadas por código
3	Consultar por código	`GET /materias/PRG4`	200	También funciona con prg4
4	Consultar código inexistente	`GET /materias/XYZ9`	404	Materia no encontrada
5	Alta con datos válidos	`POST /materias`	201	Devuelve la materia creada
6	Alta sin código	`POST /materias`	400	También rechaza " " y códigos numéricos
7	Alta sin nombre	`POST /materias`	400	También rechaza nombre con solo espacios
8	Alta sin carga horaria	`POST /materias`	400	Campo obligatorio
9	Carga horaria 0, negativa o decimal	`POST /materias`	400	Las tres variantes rechazadas
10	Código duplicado	`POST /materias`	409	Ya existe una materia con ese código
11	Estado no permitido	`POST /materias`	400	Solo activa o inactiva
12	Modificar nombre y descripción	`PUT /materias/BD1`	200	Cambio persistido
13	Modificar carga horaria y estado	`PUT /materias/BD1`	200	Cambio persistido
14	Modificar materia inexistente	`PUT /materias/XYZ9`	404	Materia no encontrada
15	Cambiar a un código ya usado	`PUT /materias/BD1`	409	Ya existe otra materia con ese código
16	Eliminar y volver a consultar	`DELETE + GET /materias/PRG3`	200 / 404	Ya no se encuentra
17	Eliminar materia inexistente	`DELETE /materias/PRG3`	404	Materia no encontrada
18	Listado final	`GET /materias`	200	Refleja altas, cambios y bajas

Error de persistencia. Antes de que la conexión quedara bien configurada (ver la sección 04), las altas fallaban en MongoDB. La API respondió con un 500 y el mensaje *"No se pudo completar la operación en la base de datos"*, sin mostrar nada interno. Así comprobé también el caso de error de conexión o persistencia que pide la consigna.

03 Frontend en React

pages/Materias.jsx NUEVO

La página reúne el formulario y la lista, y tiene los estados que los conectan, igual que en Alumnos y Docentes. `actualizar` es un contador: cada vez que una operación sale bien se incrementa, y como la lista lo tiene en las dependencias de su `useEffect`, vuelve a pedir los datos. `materiaEditar` guarda la materia elegida para editar y se usa como `key` del formulario, entonces cuando cambia React lo vuelve a montar con los datos cargados.

Lo nuevo es el estado `aviso`. Como después de guardar una edición el formulario se vuelve a montar, si el mensaje de éxito quedaba adentro del formulario se perdía. Por eso lo llevé a la página: los dos componentes avisan por props y es la página la que muestra el mensaje. La lógica del CRUD no se repite, porque el formulario guarda, la lista elimina y la página solo coordina.

frontend-react/src/pages/Materias.jsx

```jsx
import { ListaMaterias } from "../components/ListaMaterias"
import { useState } from "react"
import { Alert } from "@mui/material"
import FormularioMateria from "../components/FormularioMateria"

export default function Materias() {
const [actualizar, setActualizar] = useState(0)
const [materiaEditar, setMateriaEditar] = useState(null)
const [aviso, setAviso] = useState(null)

function operacionExitosa(mensaje) {
  setActualizar(valor => valor + 1)
  setMateriaEditar(null)
  setAviso({ tipo: "success", mensaje })
}

function mostrarError(mensaje) {
  setAviso({ tipo: "error", mensaje })
}

function seleccionarMateria(materia) {
  setMateriaEditar(materia)
  setAviso(null)
}

return (
   <>
   {aviso && (
    <Alert severity={aviso.tipo} onClose={() => setAviso(null)} sx={{ mb: 3 }}>
      {aviso.mensaje}
    </Alert>
   )}
   <FormularioMateria key={materiaEditar?.codigo ?? "nuevo"}
   onMateriaGuardada={operacionExitosa}
   materiaEditar={materiaEditar}
   onCancelar={() => setMateriaEditar(null)}
   />
   <ListaMaterias
```

```
    actualizar={actualizar}
    onMateriaEliminada={operacionExitosa}
    onEditarMateria={seleccionarMateria}
    onError={mostrarError}
    />
   </>
  )
}
```

components/FormularioMateria.jsx NUEVO

Sigue la estructura de `FormularioD`: un único estado con todos los campos, `manejarCambio` genérico por `name`, y el mismo componente sirve para dar de alta y para editar según llegue o no `materiaEditar`. Al editar, la URL del `PUT` usa el código original de la materia, así que aunque se cambie el código en el formulario la operación se identifica por el que tenía.

La función `validar` controla los datos antes de enviarlos y devuelve un objeto con un error por campo, que se muestra debajo de cada `TextField` con `helperText`. Para la carga horaria uso la expresión `/^\d+$/`, que solo acepta dígitos, y así quedan afuera los negativos y los decimales. Esta validación es para ayudar al usuario; la que manda es la del servidor. Si el backend rechaza algo, el formulario muestra el `mensaje` que vino en la respuesta. Si el error es un `TypeError`, que es lo que lanza `fetch` cuando no hay conexión, avisa que no se pudo conectar con el servidor. El estado se elige con un `TextField select` para que no se pueda escribir un valor inválido.

frontend-react/src/components/FormularioMateria.jsx

```
import { useState } from "react";
import { Alert, Box, Button, MenuItem, Paper, TextField, Typography } from "@mui/material"
const API_MATERIAS = "http://localhost:3000/materias"

const FORMULARIO_VACIO = {
    codigo: "",
    nombre: "",
    descripcion: "",
    cargaHoraria: "",
    estado: "activa"
}

function validar(formulario) {
    const errores = {}
    if (formulario.codigo.trim() === "") {
        errores.codigo = "El código es obligatorio"
    }
    if (formulario.nombre.trim() === "") {
        errores.nombre = "El nombre es obligatorio"
    } else if (formulario.nombre.trim().length < 3) {
        errores.nombre = "El nombre debe tener al menos 3 caracteres"
    }
    if (formulario.descripcion.length > 0 && formulario.descripcion.trim() === "") {
        errores.descripcion = "La descripción no puede contener solo espacios"
    }
    if (formulario.cargaHoraria.trim() === "") {
        errores.cargaHoraria = "La carga horaria es obligatoria"
    } else if (!/^\d+$/.test(formulario.cargaHoraria.trim()) || Number(formulario.cargaHoraria)
<= 0) {
        errores.cargaHoraria = "Debe ser un número entero mayor que cero"
    }
```

```
    if (!["activa", "inactiva"].includes(formulario.estado)) {
        errores.estado = "Seleccioná un estado válido"
    }
    return errores
}

function FormularioMateria({ onMateriaGuardada, materiaEditar, onCancelar }){
    const [formulario, setFormulario] = useState({
        codigo: materiaEditar?.codigo ?? "",
        nombre: materiaEditar?.nombre ?? "",
        descripcion: materiaEditar?.descripcion ?? "",
        cargaHoraria: String(materiaEditar?.cargaHoraria ?? ""),
        estado: materiaEditar?.estado ?? "activa"
    })
    const [errores, setErrores] = useState({})
    const [error, setError] = useState("")
    const [guardando, setGuardando] = useState(false)

    function manejarCambio(e) {
        const { name, value } = e.target

        setFormulario({
            ...formulario,
            [name]: value
        })
        setErrores({
            ...errores,
            [name]: ""
        })
    }

    async function guardar(e) {
        e.preventDefault()
        setError("")

        const erroresValidacion = validar(formulario)
        setErrores(erroresValidacion)
        if (Object.keys(erroresValidacion).length > 0) {
            return
        }

        setGuardando(true)

        const materia = {
            codigo: formulario.codigo.trim().toUpperCase(),
            nombre: formulario.nombre.trim(),
            descripcion: formulario.descripcion.trim(),
            cargaHoraria: Number(formulario.cargaHoraria),
            estado: formulario.estado
        }
        const editando = Boolean(materiaEditar)
        const url = editando
                    ? `${API_MATERIAS}/${encodeURIComponent(materiaEditar.codigo)}`
                    : API_MATERIAS
        try {
            const respuesta = await fetch(url, {
                method: editando ? "PUT" : "POST",
                headers: {"Content-Type":"application/json"},
                body: JSON.stringify(materia)
            })
            const datos = await respuesta.json().catch(() => null)

```

```
            if (!respuesta.ok) {
                throw new Error(datos?.mensaje || `No se pudo guardar la materia. Estado HTTP: ${respuesta.status}`)
            }

            setFormulario(FORMULARIO_VACIO)
            onMateriaGuardada(editando
                ? `Materia ${materia.codigo} modificada correctamente`
                : `Materia ${materia.codigo} registrada correctamente`)
        } catch (error) {
            console.error("Error al guardar:", error)
            setError(error instanceof TypeError
                ? "No se pudo conectar con el servidor"
                : error.message)
        } finally {
            setGuardando(false)
        }
    }
    function cancelar(){
        onCancelar()
    }
    return(
        <Paper
            component="section"
            elevation={0}
            sx={{
                p: { xs: 2.5, sm: 3.5 },
                mb: 3,
                border: 1,
                borderColor: "divider",
                borderRadius: 4,
                bgcolor: "background.paper"
            }}
        >
            <Typography component="h2" variant="h5" sx={{ mb: 2.5, fontWeight: 700 }}>
                {materiaEditar ? "Editar materia" : "Nueva Materia"}
            </Typography>
            <Box
                component="form"
                onSubmit={guardar}
                noValidate
                sx={{
                    display: "grid",
                    gridTemplateColumns: { xs: "1fr", sm: "repeat(2, minmax(0, 1fr))" },
                    gap: 2
                }}
            >
                <TextField
                    id="codigo"
                    name="codigo"
                    label="Código"
                    value={formulario.codigo}
                    onChange={manejarCambio}
                    error={Boolean(errores.codigo)}
                    helperText={errores.codigo}
                    required
                    fullWidth
                />
                <TextField
                    id="nombre"
                    name="nombre"
                    label="Nombre"
```

```
                    value={formulario.nombre}
                    onChange={manejarCambio}
                    error={Boolean(errores.nombre)}
                    helperText={errores.nombre}
                    required
                    fullWidth
                />
                <TextField
                    id="cargaHoraria"
                    name="cargaHoraria"
                    label="Carga horaria"
                    type="number"
                    value={formulario.cargaHoraria}
                    onChange={manejarCambio}
                    error={Boolean(errores.cargaHoraria)}
                    helperText={errores.cargaHoraria}
                    slotProps={{ htmlInput: { min: 1, step: 1 } }}
                    required
                    fullWidth
                />
                <TextField
                    id="estado"
                    name="estado"
                    label="Estado"
                    select
                    value={formulario.estado}
                    onChange={manejarCambio}
                    error={Boolean(errores.estado)}
                    helperText={errores.estado}
                    required
                    fullWidth
                >
                    <MenuItem value="activa">Activa</MenuItem>
                    <MenuItem value="inactiva">Inactiva</MenuItem>
                </TextField>
                <TextField
                    id="descripcion"
                    name="descripcion"
                    label="Descripción"
                    value={formulario.descripcion}
                    onChange={manejarCambio}
                    error={Boolean(errores.descripcion)}
                    helperText={errores.descripcion}
                    multiline
                    minRows={2}
                    fullWidth
                    sx={{ gridColumn: "1 / -1" }}
                />
                <Box
                    sx={{
                        gridColumn: "1 / -1",
                        display: "flex",
                        justifyContent: "flex-end",
                        flexWrap: "wrap",
                        gap: 1.5,
                        mt: 1
                    }}
                >
                    <Button type="submit" variant="contained" disabled={guardando}>
                        {guardando
                            ? "Guardando..."
                            : materiaEditar
```

```
                                ? "Guardar cambios"
                                : "Guardar"}
                    </Button>
                    {materiaEditar && (
                        <Button
                            type="button"
                            onClick={cancelar}
                            disabled={guardando}
                            variant="outlined"
                        >
                            Cancelar
                        </Button>
                    )}
                </Box>
                {error && (
                    <Alert severity="error" role="alert" sx={{ gridColumn: "1 / -1" }}>
                        {error}
                    </Alert>
                )}
            </Box>
        </Paper>
    )
}
export default FormularioMateria
```

components/ListaMaterias.jsx NUEVO

Pide las materias a la API en un `useEffect` que depende de `actualizar` y maneja tres situaciones: cargando, error de conexión y lista vacía. Por cada materia muestra el código en un `Chip`, el nombre, la carga horaria, la descripción si tiene, y el estado con un color distinto para activa e inactiva.

Para eliminar pide confirmación nombrando la materia, porque prefiero que el usuario vea exactamente qué está borrando. Si sale bien llama a `onMateriaEliminada` para que la página actualice la lista y saque la edición, por si se estaba editando justo esa materia. Si falla, en lugar de un `alert` del navegador manda el mensaje a la página con `onError`. En la URL uso `encodeURIComponent` por si algún código tuviera caracteres especiales.

frontend-react/src/components/ListaMaterias.jsx

```
import { useEffect, useState } from "react"
import { Alert, Box, Button, Chip, Paper, Stack, Typography } from "@mui/material"

const API_MATERIAS = "http://localhost:3000/materias"

export function ListaMaterias({ actualizar, onMateriaEliminada, onEditarMateria, onError }) {
    const [materias, setMaterias] = useState([])
    const [cargando, setCargando] = useState(true)
    const [error, setError] = useState("")

    async function eliminarMateria(materia){
        const confirmar = window.confirm(`¿Seguro que querés eliminar la materia
${materia.codigo} - ${materia.nombre}?`)
        if (!confirmar) {
            return
        }
        try {
            const respuesta = await
fetch(`${API_MATERIAS}/${encodeURIComponent(materia.codigo)}`,
```

```
                { method: "DELETE" }
            )
            const datos = await respuesta.json().catch(() => null)
            if (!respuesta.ok) {
                throw new Error(datos?.mensaje || `No se pudo eliminar. Estado HTTP:
${respuesta.status}`)
            }
            onMateriaEliminada(`Materia ${materia.codigo} eliminada correctamente`)
        } catch (error) {
            console.error("Error al eliminar: ", error)
            onError(error instanceof TypeError
                ? "No se pudo conectar con el servidor"
                : error.message)
        }
    }
    useEffect(() => {
        async function cargarMaterias(){
            setCargando(true)
            setError("")

            try {
                const respuesta = await fetch(API_MATERIAS)

                if (!respuesta.ok){
                    throw new Error(`No se pudo obtener la lista. Estado HTTP:
${respuesta.status}`)
                }

                const datos = await respuesta.json()
                setMaterias(datos)
            } catch(error) {
                console.error("Error de conexión", error)
                setError("No se pudo cargar la lista de materias. Verificá que el servidor esté
funcionando.")
            } finally {
                setCargando(false)
            }
        }
        cargarMaterias()
    }, [actualizar])

    if (cargando) {
        return (
            <Paper component="section" elevation={0} sx={{ p: 3, border: 1, borderColor:
"divider", borderRadius: 4 }}>
                <Typography color="text.secondary">Cargando materias...</Typography>
            </Paper>
        )
    }
    if (error) {
        return <Alert severity="error" role="alert">{error}</Alert>
    }
    return (
        <Paper
            component="section"
            elevation={0}
            sx={{
                p: { xs: 2.5, sm: 3.5 },
                border: 1,
                borderColor: "divider",
                borderRadius: 4,
                bgcolor: "background.paper"
```

```
            }}
        >
            <Typography component="h2" variant="h5" sx={{ mb: 2.5, fontWeight: 700 }}>
                Materias
            </Typography>
            {materias.length === 0
                ? (
                    <Typography color="text.secondary">
                        No hay materias registradas.
                    </Typography>
                )
                : (
                    <Stack
                        component="ul"
                        spacing={1.5}
                        sx={{ listStyle: "none", p: 0, m: 0 }}
                    >
                        {materias.map(materia => (
                            <Paper
                                component="li"
                                key={materia.codigo}
                                elevation={0}
                                sx={{
                                    p: { xs: 2, sm: 2.5 },
                                    border: 1,
                                    borderColor: "divider",
                                    borderRadius: 3,
                                    display: "flex",
                                    alignItems: { xs: "stretch", sm: "center" },
                                    justifyContent: "space-between",
                                    flexDirection: { xs: "column", sm: "row" },
                                    gap: 2
                                }}
                            >
                                <Stack
                                    direction="row"
                                    spacing={1.5}
                                    sx={{ alignItems: "center", flexWrap: "wrap" }}
                                >
                                    <Chip label={materia.codigo} color="primary"
variant="outlined" size="small" />
                                    <Box>
                                        <Typography fontWeight={600}>{materia.nombre}
</Typography>
                                        <Typography variant="body2" color="text.secondary">
                                            {materia.cargaHoraria} horas
                                            {materia.descripcion && ` · ${materia.descripcion}`}
                                        </Typography>
                                    </Box>
                                    <Chip
                                        label={materia.estado === "activa" ? "Activa" :
"Inactiva"}
                                        color={materia.estado === "activa" ? "success" :
"default"}
                                        size="small"
                                    />
                                </Stack>
                                <Stack direction="row" spacing={1} sx={{ justifyContent: "flex-
end" }}>
                                    <Button
                                        type="button"
                                        onClick={() => onEditarMateria(materia)}
```

```
                                        variant="outlined"
                                        size="small"
                                    >
                                        Editar
                                    </Button>
                                    <Button
                                        type="button"
                                        onClick={() => eliminarMateria(materia)}
                                        variant="outlined"
                                        color="error"
                                        size="small"
                                    >
                                        Eliminar
                                    </Button>
                                </Stack>
                            </Paper>
                        ))}
                    </Stack>
                )}
            </Paper>
        )
}
```

App.jsx MODIFICADO

Agregué el import de la página y la ruta `materias` dentro de `LayoutPpal`, así la página hereda el encabezado, el pie y el selector de tema como las demás. Las líneas resaltadas son las nuevas.

frontend-react/src/App.jsx

```
import Alumnos from "./pages/Alumnos"
import {FAQ} from "./pages/FAQ"
import {Routes, Route} from "react-router-dom"
import LayoutPpal from "./layouts/LayoutPpal"
import Inicio from "./pages/Inicio"
import Docentes from "./pages/Docentes"
import Materias from "./pages/Materias"


function App()
{

return (
   <Routes>
    <Route path="/" element={<LayoutPpal />}>
    <Route index element={<Inicio />} />
    <Route path="alumnos" element={<Alumnos />} />
    <Route path="docentes" element={<Docentes /> } />
    <Route path="materias" element={<Materias />} />
    <Route path="ayuda" element={<FAQ />} />
    </Route>
  </Routes>
  )
}
export default App
```

components/Header.jsx MODIFICADO

Sumé la opción Materias al menú, después de Docentes y con los mismos estilos. Es un `Link` de React Router, así que la navegación no recarga la aplicación. El resto del archivo no cambió.

frontend-react/src/components/Header.jsx *fragmento agregado*

```
<Link
    component={RouterLink}
    to="/materias"
    color="text.primary"
    underline="none"
    sx={{ px: { xs: 1, sm: 1.5 }, py: 1, borderRadius: 2, "&:hover": { bgcolor: "action.hover" } }}
>
    Materias
</Link>
```

04 Decisiones y problemas

Lo que decidí

- **El código se guarda en mayúsculas.** La consigna pide que el código identifique la materia sin ambigüedad. Si se distinguían mayúsculas, `prg4` y `PRG4` podían convivir como dos materias y eso no tiene sentido.
- **409 para los duplicados.** Los otros módulos responden 400 cuando el legajo o el DNI ya existen. Para Materias usé 409 (Conflict), que es más preciso: los datos son válidos, pero chocan con algo que ya existe. El formato de la respuesta (`{ mensaje }`) es el mismo que en el resto del backend.
- **El PUT valida todos los campos.** Lo tomé como un reemplazo completo de la materia, que es lo que hace el formulario, así que se aplican las mismas reglas que en el alta.
- **Validación en dos lados.** En el frontend para avisar rápido y marcar el campo con error. En el backend porque es la única que no se puede saltear, por ejemplo con Rest Client.

Lo que me pasó

La primera vez que probé las altas, todas devolvían 500. En la consola del servidor estaba el motivo: *"db already exists with different case"*. Mi `.env` apuntaba a la base `sga`, pero en MongoDB ya existía una llamada `SGA`, y Mongo no permite dos bases cuyo nombre solo cambia en mayúsculas. Las consultas andaban porque leer una base vacía no da error, pero cualquier escritura fallaba. Lo resolví cambiando la URI a `SGA`. Ese mismo error también afectaba las altas de Alumnos y Docentes.

Me sirvió para ver que el manejo de errores funcionaba como yo quería: el usuario recibió un mensaje controlado y el detalle quedó solo en la consola del servidor.

Valentino Serafini · Programación IV